

Лабораторная работа №22. Интерактивные веб-приложения на JavaScript

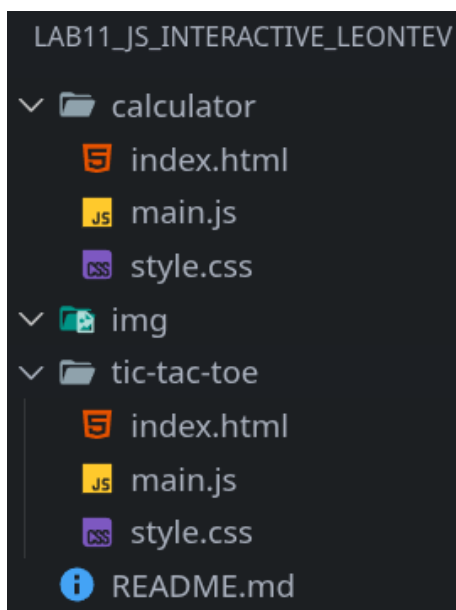
Цель лабораторной работы: Закрепить и систематизировать знания по JavaScript путём разработки интерактивных веб-приложений с использованием HTML, CSS и JavaScript, а также закрепить навыки работы с системой контроля версий Git и удалёнными репозиториями GitHub.

Шаг 1. Подготовка проекта

1. Откройте **Visual Studio Code**
2. Откройте встроенный терминал
3. Перейдите в каталог с документами
4. Создайте директорию для лабораторной работы:

Lab22_JS_Interactive_FIO (где FIO — ваша фамилия)

5. Перейдите в созданную директорию
6. Создайте структуру проекта:



Папки:

- *calculator*
- *tic-tac-toe*
- *img*

Файлы:

- *README.md*

Файлы (внутри папок):

- *index.html*
- *style.css*
- *main.js*

7. Откройте созданную папку в **Visual Studio Code**

Инициализация Git-репозитория

git init

git add .

git commit -m "Initial commit"

Создание удалённого репозитория

1. Создайте **пустой public-репозиторий** на GitHub с именем:

Lab22_JSInteractive

2. Привяжите локальный репозиторий к удалённому:

git branch -M main

git remote add origin <URL_репозитория>

git push -u origin main

4. Сделайте **скриншот терминала** с выполненными командами:

➤ **commit**

➤ **remote add**

➤ **push**

5. В папке **img** сохраните изображение удачного пуша под именем:

gitPushLab22_JS_Interactive_FIO.png

6. Добавьте изменения и отправьте их:

git add .

git commit -m "Added push screenshot and project update"

git push

Проект. Калькулятор

Шаг 2. Создание HTML-разметки калькулятора

1. Откройте файл **calculator/index.html**

2. Создайте базовую HTML-структуру документа

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Калькулятор</title>
  </head>
  <body></body>
</html>
```

3. Добавьте в **<body>** контейнер калькулятора — **div**. Он нужен, чтобы управлять стилями и логикой одного компонента. Позже будем обращаться к нему из JS

```
<body>
+ <div class="calculator"></div>
</body>
```

4. Подключим **базовые стили** для калькулятора и **JS**:

```

<title>Калькулятор</title>
+<link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="calculator"></div>
+ <script src="main.js"></script>
</body>

```

5. Внутри контейнера добавьте поле для вывода выражения и результата:

```

<div class="calculator">
+ <input type="text" id="display" readonly />
</div>

```

Когда к тегу **<input>** добавляется атрибут **readonly**, текстовое поле не может изменяться пользователем.

6. Далее внутрь контейнера добавим кнопки калькулятора:

```

<div class="calculator">
  <input type="text" id="display" re
  <div class="buttons">
    <button>7</button>
    <button>8</button>
    <button>9</button>
    <button>/</button>

    <button>4</button>
    <button>5</button>
    <button>6</button>
    <button>*</button>

+ <button>1</button>
    <button>2</button>
    <button>3</button>
    <button>-</button>

    <button>0</button>
    <button>.</button>
    <button id="clear">C</button>
    <button>+</button>

    <button class="equal">=</button>
  </div>
</div>

```

- цифры
- операции
- кнопку очистки
- кнопку равно

Важно:

- кнопки создаются явно, не циклом
- без классов и JS-логики

Практическое задание

1. Сделайте скриншот страницы с результатом работы в папку **img** с именем:

step2_calculatorHtmlLab22_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 2: added HTML structure for calculator"

git push

Шаг 3. Базовые стили калькулятора

Откройте файл `calculator/style.css`

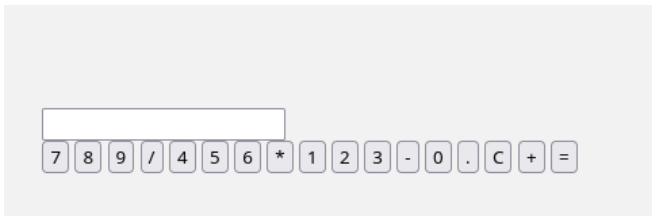
3.1. Стили для всего документа (тег `body`):

```
1 body {
2   font-family: Arial, sans-serif;
3   background: #f2f2f2;
4   display: flex;
5   justify-content: center;
6   align-items: center;
7   height: 100vh;
8 }
```

font-family: Основной шрифт
background: Фон страницы
display: Включаем гибкую вёрстку (Flexbox)
justify-content: Выравниваем содержимое по горизонтали по центру
align-items: Выравниваем содержимое по вертикали по центру
height: Задаём высоту тела равной 100% высоты окна браузера

Что делает:

- **Сбрасывает** стандартный шрифт браузера на Arial.
- Задаёт странице **светлый нейтральный фон**.
- С помощью **display: flex** и свойств выравнивания **центрирует** всё содержимое страницы (наш калькулятор) как по горизонтали, так и по вертикали.
- **height: 100vh** гарантирует, что центрирование работает на всю высоту экрана.



Визуальный результат:

После применения этих стилей калькулятор будет расположен точно в центре экрана на светло-сером фоне.

3.2. Стили основного блока калькулятора (`.calculator`)

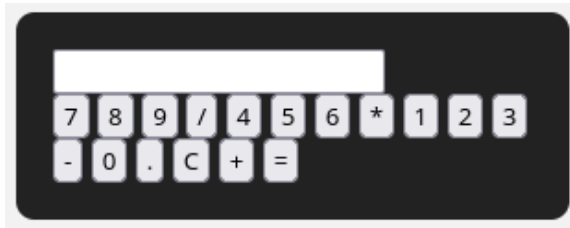
```
10 .calculator {
11   background: #222;
12   padding: 20px;
13   border-radius: 10px;
14   width: 260px;
15 }
```

background: Тёмно-серый фон
padding: Внутренние отступы со всех сторон
border-radius: Закругление углов на 10 пикселей
width: Фиксированная ширина блока

Что делает:

- Создаёт **тёмный фон** для контраста с фоном страницы и кнопками.
- Добавляет **воздуха внутри** блока с помощью `padding`, чтобы элементы не прижимались к краям.
- **Скругляет углы**, придавая калькулятору современный вид.

- Фиксирует **ширину** контейнера.



Визуальный результат:

На экране появится тёмный прямоугольник с закруглёнными углами и отступами внутри.

3.3. Стили поля ввода (дисплея) (#display)

```
17 #display {
18   width: 100%;
19   height: 50px;
20   font-size: 24px;
21   margin-bottom: 10px;
22   text-align: right;
23   padding-right: 10px;
24 }
```

width: Занимает 100% ширины родителя

height: Фиксированная высота

font-size: Крупный размер шрифта

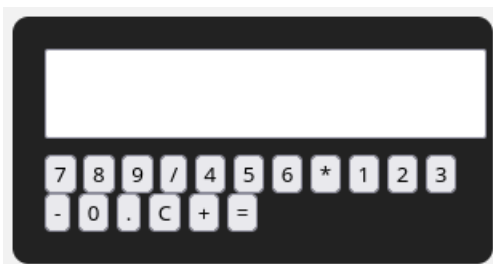
margin-bottom: Отступ снизу, чтобы отделить от кнопок

text-align: Выравнивание текста по правому краю

padding-right: Отступ текста от правого края поля

Что делает:

- Растягивает поле на **всю доступную ширину**.
- Задаёт **достаточную высоту** для удобного отображения чисел.
- Устанавливает **крупный шрифт** для лучшей читаемости.
- **Отодвигает** дисплей от сетки кнопок с помощью **margin-bottom**.
- **Выравнивает вводимые числа по правому краю**, как в настоящем калькуляторе, и добавляет небольшой отступ от границы.



Визуальный результат:

В верхней части тёмного блока появится большое белое (по умолчанию) поле ввода с текстом, выровненным по правому краю.

3.4. Стили контейнера для кнопок (.buttons)

```
26 .buttons {
27   display: grid;
28   grid-template-columns: repeat(4, 1fr);
29   gap: 10px;
30 }
```

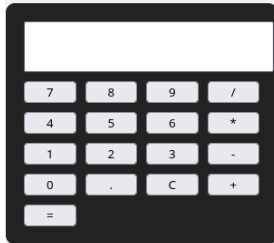
display: Включаем вёрстку на основе CSS Grid

grid-template-columns: Создаём 4 колонки равной ширины

gap: Задаём расстояние между ячейками сетки (кнопками)

Что делает:

- **display: grid** создаёт **сетчатую структуру** для расположения кнопок.
- **grid-template-columns: repeat(4, 1fr)** — ключевое свойство. Оно автоматически создаёт 4 колонки (4). 1fr означает, что каждая колонка займёт **одну долю (fraction)** доступного пространства, делая их **одинаковыми по ширине**.
- **gap** добавляет **фиксированные промежутки** между кнопками, делая интерфейс аккуратным.



Визуальный результат:

Под дисплеем появится невидимая сетка 4x4, готовая к размещению кнопок. Пока что она пуста.

3.5. Базовые стили для всех кнопок (button)

```
32 button {  
33   height: 50px;  
34   font-size: 18px;  
35   cursor: pointer;  
36 }
```

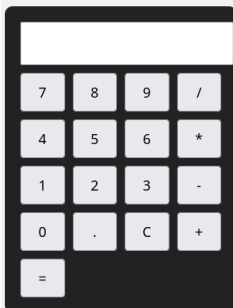
height: Высота, равная высоте дисплея

font-size: Размер шрифта на кнопках

cursor: Меняет курсор на "указатель" при наведении

Что делает:

- Задаёт **одинаковую высоту** для всех кнопок, создавая единый визуальный ритм.
- Устанавливает **удобочитаемый размер** цифр и символов на кнопках.
- Меняет вид курсора мыши, давая пользователю **визуальный сигнал**, что элемент кликабелен.



Визуальный результат:

Все 16 кнопок (4x4) станут одного размера и будут реагировать на наведение курсора.

3.6. Стили для специальной кнопки "Равно" (.equal)

```
38 .equal {  
39   grid-column: span 4;  
40   background: orange;  
41 }
```

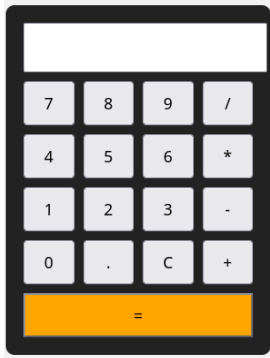
grid-column: Растягивает элемент на 4 колонки сетки

background: Задаёт оранжевый цвет фона

Что делает:

➤ **grid-column: span 4** — самое важное свойство. Оно заставляет эту кнопку **растянуться на все 4 колонки** созданной ранее сетки, делая её **во всю ширину** калькулятора.

➤ **background: orange** **визуально выделяет** самую важную кнопку другим цветом.



Визуальный результат:

Кнопка "=" (equal) станет в 4 раза шире обычной кнопки и будет окрашена в оранжевый цвет, занимая всю последнюю строку калькулятора.

Практическое задание

1. Сделайте скриншот оформленного калькулятора в папку **img** с именем:

step3_calculatorCssLab22_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 3: added basic CSS styling for calculator"

git push

Шаг 4. Подключение JavaScript

Откройте файл **calculator/main.js**

4.1. Получение элементов DOM

```
1 const display = document.getElementById("display");
2 const buttons = document.querySelectorAll("button");
3 const clearBtn = document.getElementById("clear");
```

Что делает:

➤ **display** — получает ссылку на поле ввода (дисплей) калькулятора по его ID для отображения вычислений.

➤ **buttons** — собирает все кнопки калькулятора в одну коллекцию (NodeList) для последующей обработки.

➤ **clearBtn** — получает ссылку на кнопку очистки (можно использовать, хотя в текущей логике обрабатывается вместе со всеми кнопками).

4.2. Инициализация переменной состояния

```
4 let expression = "";
```

Что делает:

➤ Создаёт **переменную-аккумулятор** **expression**, которая будет хранить текущее математическое выражение.

➤ Изначально устанавливается как **пустая строка**, что соответствует чистому дисплею калькулятора.

Как работает: При нажатии кнопок "5", "+", "3" в **expression** последовательно запишется строка: "5+3".

4.3. Навешивание обработчиков событий на все кнопки

```
6 buttons.forEach((button) => {
7   button.addEventListener("click", () => {
8     // Логика обработки клика
9   });
10 });
```

Что делает:

➤ **forEach** перебирает **каждую кнопку** из полученной коллекции.

➤ Для каждой кнопки добавляется **обработчик события click** — функция, которая выполнится при нажатии на кнопку.

➤ Используется **стрелочная функция** для краткости синтаксиса.

Визуальный результат:

Теперь каждая кнопка калькулятора "оживает" — готова реагировать на клики пользователя.

4.4. Логика обработки нажатия кнопки

```
6 buttons.forEach((button) => {
7   button.addEventListener("click", () => {
8     + const value = button.textContent;
```

Что делает:

➤ При каждом клике получает **текстовое содержимое** нажатой кнопки (цифру, оператор или символ).

➤ Сохраняет его в переменную **value** для дальнейшего анализа.

Пример: Если нажата кнопка с цифрой "7", то `value = "7"`.

4.5. Обработка кнопки очистки ("C")

```
8      const value = button.  
9      if (value === "C") {  
10         expression = "";  
11         display.value = "";  
12         return;  
13     }
```

Что делает:

1. **Проверяет**, была ли нажата кнопка "C" (Clear).

2. Если да:

➤ **Сбрасывает** аккумулятор **expression** в пустую строку.

➤ **Очищает** видимое поле ввода (дисплей).

➤ **Выходит** из функции досрочно с помощью **return**, чтобы не выполнять оставшийся код.

Результат: Дисплей и внутреннее состояние полностью очищаются, калькулятор ГОТОВ К НОВЫМ ВЫЧИСЛЕНИЯМ.

4.6. Обработка кнопки вычисления ("=")

```
14     if (value === "=") {  
15         try {  
16             expression = eval(expression);  
17             display.value = expression;  
18         } catch {  
19             display.value = "Ошибка";  
20             expression = "";  
21         }  
22         return;  
23     }
```

Что делает:

1. **Проверяет**, была ли нажата кнопка "=".

2. Если да, пытается **вычислить** накопленное выражение:

➤ **eval(expression)** — встроенная функция JavaScript, которая вычисляет строку как математическое выражение (например, `eval("2+2")` вернёт 4).

- **try...catch** — конструкция для обработки возможных ошибок.

3. При успешном вычислении:

- Результат записывается обратно в **expression** (позволяет продолжать вычисления с результатом).

- Результат выводится на дисплей.

4. При ошибке (неверное выражение, например "5++3"):

- На дисплее появляется "Ошибка".
- Внутреннее состояние сбрасывается.

5. Выходит из функции после выполнения.

Примеры работы:

"5+3" → на дисплее 8

"10/0" → на дисплее "Ошибка"

4.7. Обработка цифр и операторов (основной поток)

```
24     expression += value;  
25     + [display.value = expression;  
26     }];  
27     });
```

Что делает:

- Добавляет символ нажатой кнопки (цифру, оператор, точку) к текущему выражению.
- Сразу отображает обновлённое выражение на дисплее.

Когда выполняется:

Для всех кнопок, которые не "C" и не "=" (цифры 0-9, операторы + - * /, точка).

Практическое задание

1. Сделайте скриншот оформленного калькулятора в папку **img** с именем:

step4_calculatorJsLab22_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 4: added basic JS code for calculator"

git push

Шаг 5. Рефакторинг кода калькулятора

Какие у нас имеются проблемы в текущем коде:

1. Архитектурные проблемы:

- **Монолитный код** — весь код в одном файле (main.js)
- **Смешивание ответственности** — логика вычислений, обработка событий и управление DOM в одной функции
- **Сложность поддержки** — при добавлении новых функций код станет нечитаемым

2. Функциональные проблемы:

- **Уязвимость безопасности** — использование eval() без проверок
- **Некорректный ввод** — можно ввести 5++3 или 5.2.1
- **Отсутствие валидации** — нет проверки на корректность математических выражений

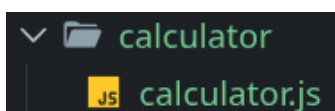
3. Организационные проблемы:

- **Глобальные переменные** — expression доступна извне
- **Повторяющийся код** — нет переиспользуемых функций
- **Сложность тестирования** — логика не отделена от DOM

Цели рефакторинга:

1. **Разделить ответственность** — вынести логику калькулятора в отдельный класс
2. **Повысить безопасность** — добавить валидацию выражений
3. **Улучшить UX** — предотвратить некорректный ввод
4. **Сделать код поддерживаемым** — легко добавлять новые функции

5.1. Создаём отдельный файл с классом calculator.js



Переходим к написанию кода в **calculator.js**

5.2. Создание класса Calculator

```
1 export class Calculator {
2   constructor() {
3     this.display = document.getElementById("display");
4     this.expression = "";
5     this.hasError = false;
6   }
7 }
```

- создаём отдельный класс **Calculator**
- в конструкторе:

- ✓ получаем ссылку на DOM-элемент отображения
- ✓ создаём переменную `expression` для хранения текущего выражения
- ✓ флаг `hasError` показывает, находится ли калькулятор в состоянии ошибки

Аналогия с C#:

- `constructor` = конструктор класса
- поля класса = поля объекта

5.3. Метод `init()`

Обратите внимание, далее весь код мы пишем внутри класса.

```
6  }
7
8  init() {
9    document.querySelectorAll("button").forEach((button) => {
10     button.addEventListener("click", () => {
11       this.handleButtonClick(button.textContent);
12     });
13   });
14   this.updateDisplay();
15 }
```

- инициализирует калькулятор
- связывает кнопки с логикой
- выполняется один раз при запуске

5.4. Метод `handleButtonClick(value)`

```
17 handleButtonClick(value) {
18   if (this.hasError && value !== "C") {
19     this.clear();
20   }
21   if (value === "C") {
22     this.clear();
23   } else if (value === "=") {
24     this.calculate();
25   } else {
26     this.addToExpression(value);
27   }
28 }
```

Метод отвечает за **обработку нажатия кнопки** и принимает:

- **value** — текст кнопки

Логика работы

1. Если ранее была ошибка и нажата любая кнопка, кроме C — калькулятор очищается
2. Если нажата кнопка:

- $C \rightarrow$ очистка
- $= \rightarrow$ вычисление
- иначе $\rightarrow$ добавление символа в выражение

Ключевая идея: метод не считает, а распределяет действия.

5.5. Метод `addToExpression(value)`

```

30 addToExpression(value) {
31   if (this.isOperator(value) && this.isLastCharOperator()) {
32     this.expression = this.expression.slice(0, -1) + value;
33   } else if (value === "." && this.hasDuplicateDot()) {
34     return;
35   } else if (this.isOperator(value) && !this.expression && value !== "-") {
36     return;
37   } else {
38     this.expression += value;
39   }
40   this.updateDisplay();
41 }

```

Метод добавляет символ в текущее выражение, проверяя корректность ввода.

Что здесь проверяется:

- нельзя вводить два оператора подряд
- нельзя вводить несколько точек в одном числе
- выражение не может начинаться с оператора (кроме -)
- корректное выражение сразу отображается на экране

Это пример валидации ввода на клиенте.

5.6. Метод `calculate()`

```

43 calculate() {
44   if (!this.expression || this.hasError) return;
45   try {
46     const result = this.safeEvaluate(this.expression);
47     if (!isFinite(result)) {
48       throw new Error("Некорректный результат");
49     }
50     this.expression = String(result);
51     this.hasError = false;
52     this.updateDisplay();
53   } catch (error) {
54     this.showError("Ошибка");
55   }
56 }

```

- вычисляет выражение
- обрабатывает ошибки:
- ✓ проверяется деление на ноль

- ✓ при ошибке калькулятор не падает
- ✓ пользователь видит сообщение об ошибке
- обновляет состояние калькулятора

5.7. Метод `safeEvaluate(expr)`

```

58 safeEvaluate(expr) {
59   if (!this.isValidExpression(expr)) {
60     + throw new Error("Некорректное выражение");
61   }
62   return eval(expr);
63 }

```

- проверяет корректность выражения
- изолирует использование `eval`
- повышает читаемость и безопасность

5.8. Метод проверки выражения `isValidExpression()`

```

65 isValidExpression(expr) {
66   expr = expr.replace(/\s+/g, ""); // Удаляем пробелы
67   // Не должно быть двух операторов подряд (кроме минуса в начале числа)
68   const doubleOps = /[+\-*/]{2,}/;
69   + if (doubleOps.test(expr)) return false;
70   // Не должно быть оператора в конце
71   if (/[\+ \-*/]$/.test(expr)) return false;
72   return true;
73 }

```

- проверяет отсутствие двойных операторов
- запрещает оператор в конце строки

5.9. Метод очистки `clear()`

```

75 ✓ clear() {
76   this.expression = "";
77   this.hasError = false;
78   this.updateDisplay();
79 }

```

- сбрасывает состояние калькулятора

5.10. Обновление экрана `updateDisplay()`

```

81 updateDisplay() {
82   if (this.hasError) return;
83   this.display.value = this.expression || "0";
84 }

```

- обновляет поле вывода
- показывает 0, если выражение пустое

5.11. Метод отображения ошибки showError()

```
86   showError(message) {  
87     this.display.value = message;  
88     this.hasError = true;  
89     + this.expression = "";  
90   }
```

- переводит калькулятор в состояние ошибки

5.12. Метод проверки оператора на символ isOperator()

```
92   isOperator(char) {  
93     return ["+", "-", "*", "/"].includes(char);  
94   }
```

5.13. Метод проверки последнего оператора на символ isLastCharOperator()

```
96   isLastCharOperator() {  
97     if (!this.expression) return false;  
98     const lastChar = this.expression.slice(-1);  
99     return this.isOperator(lastChar);  
100  }
```

5.14. Метод проверки есть ли уже точка в текущем числе hasDuplicateDot()

```
102  hasDuplicateDot() {  
103    const parts = this.expression.split(/[+|-|*|\/|]/);  
104    const lastNumber = parts[parts.length - 1];  
105    return lastNumber && lastNumber.includes(".");  
106  }
```

5.15. Создаём и запускаем калькулятор при загрузке страницы.

Вносим изменения в файл **main.js**:

```
1  import { Calculator } from "../calculator.js";  
2  
3  document.addEventListener("DOMContentLoaded", () => {  
4    const calc = new Calculator();  
5    calc.init();  
6  });
```

Финальная структура класса Calculator

Вот как выглядит полный класс после всех изменений:

```
1  export class Calculator {
2  >  .. constructor() { ...
6    .. }
7
8  >  .. init() { ...
15   .. }
16
17 >  .. handleClick(value) { ...
28   .. }
29
30 >  .. addToExpression(value) { ...
41   .. }
42
43 >  .. calculate() { ...
56   .. }
57
58 >  .. safeEvaluate(expr) { ...
63   .. }
64
65 >  .. isValidExpression(expr) { ...
73   .. }
74
75 >  .. clear() { ...
79   .. }
80
81 >  .. updateDisplay() { ...
84   .. }
85
86 >  .. showError(message) { ...
90   .. }
91
92 >  .. isOperator(char) { ...
94   .. }
95
96 >  .. isLastCharOperator() { ...
100  .. }
101
102 >  .. hasDuplicateDot() { ...
106  .. }
107 }
```

5.16. Вносим изменения в index.html для поддержки ES6 модулей

```
<script type="module" src="main.js"></script>
```

Что такое ES6 модули?

Преимущества модулей:

- **Разделение кода:**

calculator.js → Класс Calculator (логика)

main.js → Инициализация (запуск)

- **Переиспользование:**

Можно использовать Calculator в других проектах!

- **Изоляция:**

Переменные из calculator.js не "утекают" в глобальную область

- **Современный стандарт:**

Так работают React, Vue, Angular

⚠ **Важно:** Атрибут `type="module"` **обязателен**, иначе браузер не поймёт синтаксис `import/export`.



Вывод: Модули помогают организовать код, как пространства имён в C#.

Практическое задание

1. Сделайте скриншот оформленного калькулятора в папку **img** с именем:

step5_calculatorJs_Redactoring_Lab22_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 5: refactoring code for calculator"

git push

Самостоятельное задание: создание README.md для проекта

Задание: Добавьте описание в README.md.

1. Описание проекта.

2. Функциональность:

- Какие операции поддерживает?
- Какие есть ограничения?
- Как обрабатываются ошибки?

3. Архитектура:

- Объяснить структуру файлов
- Описать класс Calculator
- Схема взаимодействия компонентов

4. Логика работы:

- Последовательность действий при нажатии кнопок
- Как работает валидация ввода?
- Как происходит вычисление?